

ML-Guided Beam Search for Differential Trail Discovery in SPN Ciphers: A Case Study on GIFT-64

Alireza Gholizadeh Shahrbejari*

Department of Computer Engineering, University of Guilan, Rasht, Iran

Reza Ebrahimi Atani†

Computer Security Incident Response Teams (CSIRT) Research Laboratory,
Department of Computer Engineering, University of Guilan, Rasht, Iran

December 6, 2025

Abstract

This paper introduces an ML-guided scoring heuristic for differential trail beam search in substitution-permutation network (SPN) ciphers. Instead of replacing classical search procedures or relying on heavy learning architectures, we take a residual-learning approach: a gradient boosting regressor is trained to predict the error of a simple nibble-count lower bound on the remaining trail cost. At search time, the predicted residual is fused multiplicatively into the beam scoring function, using per-layer robust normalization and a conservative floor to preserve safety. This design keeps the underlying search structure such as beam width, pruning rules, and lower-bound guarantees unchanged, while aiming to improve the ranking of partial trails. We instantiate the method on the 64-bit block cipher GIFT-64 under the classical Markov differential model. Our implementation reproduces state of the art differential trails with identical weights and round by round differences, and achieves 10–40% reductions in the number of expanded nodes in moderate-depth searches, with runtime trade-offs analyzed across different model horizons. The results suggest a practical, non-invasive paradigm for enhancing classical cryptanalytic search with learned corrections, without redesigning existing algorithms or probability models, and are in principle applicable to a range of SPN designs.

Keywords: Differential cryptanalysis, Beam search, Machine learning, GIFT-64, Residual learning, SPN ciphers, Automated search.

1 Introduction

Differential cryptanalysis studies how input differences propagate through a cipher and with what probability they produce specific output differences [3, 17]. In Substitution-Permutation Network (SPN) ciphers with 4-bit S-boxes, security evaluations routinely hinge on discovering low-weight differential *trails* (under the classical Markov assumption) or proving tight lower bounds on trail

*Email: gholizadeh.a2000@gmail.com

†Email: rebrahimi@guilan.ac.ir

weights across many rounds. Exact branch-and-bound (BnB) can certify optimality but often struggles to scale due to global queues and bound maintenance; in practice, fixed-width *beam search* is widely used for its predictable memory footprint and strong throughput, with effectiveness governed by the *scoring function* that ranks partial trails at each depth [24].

We propose a small, compatible change to the scoring step: instead of using only a cheap round-wise surrogate for the remaining cost (e.g., a linear function of active nibbles times the residual horizon), we learn a *residual correction* to that surrogate and fuse it multiplicatively and robustly into the beam score. The correction improves *relative* ordering while guarding against outliers. Importantly, we do *not* change beam width, α -pruning, lifeboats, or any other pruning rule; only the tail estimate inside the existing score is replaced.

A long line of work searches for high-probability differential *trails* by assuming round-independence and composing per-S-box DDT probabilities. On the algorithmic side, Yeoh et al. design a GPU-accelerated BnB that preserves optimality while greatly improving throughput for automated differential search [34]. For bit-permutation-based SPNs (e.g., PRESENT/RECTANGLE/GIFT), Weng et al. further refine Matsui-style search with divide-and-conquer-style decompositions and pruning/ordering tweaks, reporting best differential/linear trails under the same Markov model [33]. In parallel, mixed-integer linear programming (MILP) has become a standard vehicle for encoding DDT transitions and linear layers of SPNs: Mouha et al. formulate differential/linear cryptanalysis as MILP to derive tight bounds and search for characteristics [25]; Sun et al. automate security evaluation for bit-oriented ciphers (including PRESENT and related-key variants) within the same framework [32]. Scalability has been improved further by divide-and-conquer MILP strategies [35] and by practical MILP-driven differential attacks on round-reduced *GIFT* [36]. Modeling advances also extend to addition-heavy designs via MILP encodings for modular additions [15]. Beyond MILP, SAT encodings provide an alternative solver back-end for trail search and experimental verification while adhering to DDT-style costs, as shown for Midori64 by Li and Wang [20]. All these works remain in the classical Markov setting and primarily target the *search/encoding layer*: they accelerate exploration or strengthen modeling under a fixed probability model.

For the concrete case of *GIFT-64*, several dedicated differential, linear, and rectangle attacks have been proposed on top of these generic tools. Table 1 summarizes representative cryptanalytic results (in chronological order); “KR” stands for single-key key-recovery attacks and “RK” for related-key attacks.

Table 1: Representative cryptanalytic results on GIFT-64.

Work	Rounds / setting	Main result on GIFT-64
Banik et al. [1]	Design, no KR attack	Original specification of GIFT-64/128; lightweight hardware-oriented SPN with differential/linear bounds and round-reduced security analysis.
Zhu et al. [36]	19-round SK differential KR	MILP-based search for differential characteristics; best 12-round characteristic and iterative 4-round trails, leading to a 19-round single-key differential key-recovery attack.
Chen et al. [8]	20-/21-round SK differential KR	Automatic search for multiple high-probability differentials; 20-round attack with 2^{62} chosen plaintexts and a 21-round single-key attack using the full codebook.
Chen et al. [9]	23-round RK rectangle KR	MILP-aided related-key rectangle cryptanalysis; construction of RK boomerang/rectangle distinguishers and a 23-round related-key rectangle attack on GIFT-64.
Ji et al. [16]	up to 25-round RK attacks	Improved related-key differential and boomerang trails; revisits previous RK analysis and extends RK rectangle-type attacks on reduced-round GIFT-64.
Sun et al. [31]	20-round SK diff., 19-round lin.	Refined single-key differential characteristics and extended linear cryptanalysis under the classical Markov model on GIFT-64.
Cui et al. [11]	16-round linear attack	MILP-based modeling of linear trails; finds 11-round linear characteristics and derives a 16-round linear attack on GIFT-64.
Cui et al. [12]	19-round linear attack	Use of equivalent linear trails and multiple linear approximations to push linear attacks on GIFT-64 to 19 rounds.
Chen et al. [10]	26-round RK rectangle KR	Improved MILP modeling and key-guessing strategies that reduce the complexity of 26-round related-key rectangle attacks on GIFT-64.
Canale & Naya-Plasencia [6]	up to 26-round RK differential KR	Generic “guessing less and better” framework; sharpens key-recovery phases and improves time/memory of the best RK differential attacks on GIFT-64.

A substantial and complementary literature revisits the validity and key dependence of differential characteristics. Daemen and Rijmen show *plateau characteristics* for AES, where a characteristic has probability zero for many keys and a fixed non-zero value for the remaining ones [13]. Subsequent works develop affine-space and constraint-based formalisms, SMT/MILP encodings, and quasidifferential trails to detect impossible characteristics, derive multi-bit/key conditions, and obtain key-dependent probability estimates [19, 14, 7, 30, 22, 21, 2, 27, 29, 26]. These methods refine the *probability model* itself and offer tools to validate or reweight given characteristics; they are orthogonal to faster trail search per se.

Our Contribution We operate entirely within the classical Markov trail framework: round probabilities are modeled via S-box DDTs and trail weights are sums of $-\log_2 p$ over active S-boxes [3, 17]. Our goal is orthogonal to key-dependent analyses: given this widely used model and

Table 2: GIFT 4-bit S-box $GS : \{0, \dots, 15\} \rightarrow \{0, \dots, 15\}$ [1].

x	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
$GS(x)$	1	a	4	c	6	f	3	9	2	d	b	7	5	0	8	e

fixed beam/pruning parameters, we improve the *efficiency* of search by learning a better heuristic for ranking partial trails. Trails discovered by our method can, in principle, be post-processed by key-dependent tools to assess validity and refine probabilities, but designing or improving such tools lies outside our scope.

Organization The remainder of this paper is organized as follows. Section 2 reviews the structure of GIFT-64, recalls differential trails under the classical Markov model, and introduces the cost metrics and search paradigms we build upon. Section 3 describes our ML-guided residual scoring method and its integration into a standard beam search. Section 4 presents experimental results on GIFT-64, including reproduction of published trails and an ablation over depths and model horizons. Section 5 concludes and outlines directions for future work.

2 Preliminaries

This section recalls the structure of GIFT-64, basic notions on differential trails under the classical Markov model, the cost metrics used to evaluate trails and searches, and the search paradigms we build upon.

2.1 GIFT-64

GIFT is a family of lightweight block ciphers built on the Substitution–Permutation Network (SPN) structure, available in two main variants: GIFT-64 and GIFT-128. Both use a 128-bit master key, but differ in block size and number of rounds. GIFT-64 operates on 64-bit blocks over 28 rounds, whereas GIFT-128 works on 128-bit blocks over 40 rounds [1]. Compared to contemporaries such as PRESENT [5], GIFT-64 offers improved hardware efficiency and strong resistance to classical differential and linear cryptanalysis.

Each round of GIFT applies three core transformations: *SubCells*, *PermBits*, and *AddRoundKey*. We adopt the bit-indexing convention of the specification: the state bits are indexed from b_0 (LSB) to b_{63} (MSB), hence $n = 64$ for GIFT-64.

SubCells. A 4-bit nonlinear S-box, denoted $GS(x)$, is applied in parallel to all 16 nibbles of the state. The mapping is shown in Table 2.

PermBits. The bit-permutation layer relocates each bit according to a fixed pattern $P_{64}(i)$, defined by [1]

$$P_{64}(i) = 4 \left\lfloor \frac{i}{16} \right\rfloor + 16 \left(\left(3 \left\lfloor \frac{i \bmod 16}{4} \right\rfloor + (i \bmod 4) \right) \bmod 4 \right) + (i \bmod 4). \quad (1)$$

This mapping achieves fast diffusion in a hardware-friendly manner. The explicit table (often used for implementation) appears in Table 3.

Table 3: GIFT-64 bit-permutation table $P_{64}(i)$ [1].

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$P_{64}(i)$	0	17	34	51	48	1	18	35	32	49	2	19	16	33	50	3
i'	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
$P_{64}(i')$	4	21	38	55	52	5	22	39	36	53	6	23	20	37	54	7
i	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
$P_{64}(i)$	8	25	42	59	56	9	26	43	40	57	10	27	24	41	58	11
i'	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
$P_{64}(i')$	12	29	46	63	60	13	30	47	44	61	14	31	28	45	62	15

AddRoundKey. A 64-bit round key $RK = U \parallel V$ (two 32-bit words) is injected by XOR into selected bit positions:

$$b_{4i+1} \leftarrow b_{4i+1} \oplus u_i, \quad b_{4i} \leftarrow b_{4i} \oplus v_i, \quad \forall i \in \{0, \dots, 15\}.$$

Additionally, one constant bit and a 6-bit round constant $C = c_5 c_4 c_3 c_2 c_1 c_0$ are XORed into pre-determined positions to break symmetry:

$$\begin{aligned} b_{63} &\leftarrow b_{63} \oplus 1, \\ b_{23} &\leftarrow b_{23} \oplus c_5, \quad b_{19} \leftarrow b_{19} \oplus c_4, \quad b_{15} \leftarrow b_{15} \oplus c_3, \\ b_{11} &\leftarrow b_{11} \oplus c_2, \quad b_7 \leftarrow b_7 \oplus c_1, \quad b_3 \leftarrow b_3 \oplus c_0. \end{aligned}$$

Further details on the key schedule and constants can be found in the specification [1].

2.2 Differential Trails

A *differential trail* is a round-by-round evolution of input-output differences through the cipher, typically under the Markov assumption of round-independence [17, 3]. Let the post-round- r state difference be $\Delta^r \in \{0, 1\}^n$. In each round, an m -bit S-box layer is followed by a linear diffusion layer. If the input/output differences of the i -th S-box are $\alpha, \beta \in \{0, 1\}^m$, the differential distribution table (DDT) determines

$$\Pr[\alpha \rightarrow \beta] = \frac{\#\{x \mid S(x) \oplus S(x \oplus \alpha) = \beta\}}{2^m}. \quad (2)$$

The associated transition weight is

$$w(\alpha \rightarrow \beta) = -\log_2 \Pr[\alpha \rightarrow \beta], \quad (3)$$

and the total weight of a D -round trail $\mathcal{T}$ is

$$W(\mathcal{T}) = \sum_{r=0}^{D-1} \sum_{i=1}^{n/m} w(\Delta_i^r \rightarrow \Gamma_i^r),$$

where Γ_i^r is the output difference of S-box i in round r .

Differential distribution table (DDT). The S-box DDT lists, for each α, β , the count of solutions to $S(x) \oplus S(x \oplus \alpha) = \beta$. Figure 1 shows raw counts for the GIFT S-box ($m = 4$); Figure 2 depicts $-\log_2(\text{count}/2^m)$. These plots highlight higher-probability transitions that often dominate low-weight trails [3].

For example, for a 4-bit S-box, nonzero DDT counts lie in $\{1, 2, 4, 8, 16\}$, mapping to weights $\{4, 3, 2, 1, 0\}$. Empirically, specific (α, β) pairs (e.g., $\alpha = 0\mathbf{x}6$, $\beta = 0\mathbf{x}3$) yield lower weights and thus frequently appear in optimized trails [8].

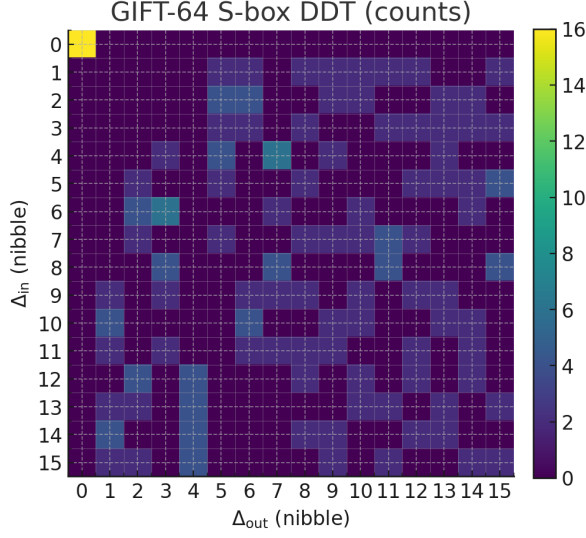


Figure 1: DDT counts for the GIFT S-box ($m = 4$). Lighter cells indicate larger counts.

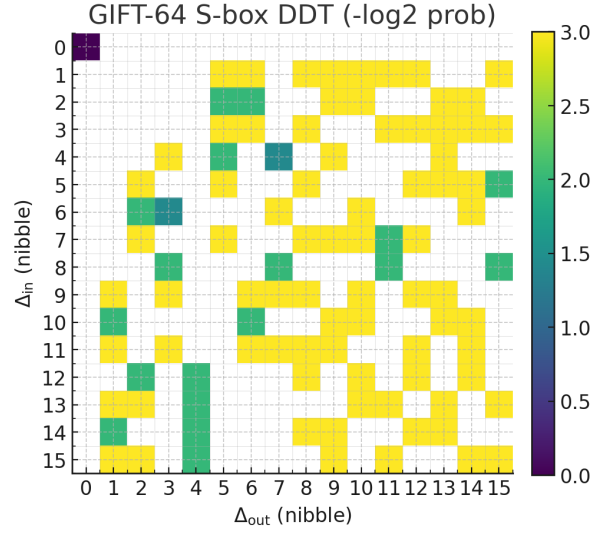


Figure 2: DDT visualized as $-\log_2 p$. Blank cells denote impossible transitions.

Trail vs. hull. A trail specifies every intermediate difference. A *differential hull*, instead, aggregates all trails sharing the same end-points (Δ^0, Δ^D) [4]:

$$\Pr[\Delta^0 \Rightarrow \Delta^D] = \sum_{\mathcal{T}: \Delta^0 \rightarrow \Delta^D} \Pr(\mathcal{T}).$$

While hull probabilities are the ultimate target for attack complexity, automated tools typically search for low-weight trails (or tight bounds) due to tractability [28, 23].

2.3 Cost Metrics

Evaluating and comparing trails requires metrics that capture both likelihood and search effort.

Negative log-probability (NegLog). The trail probability is the product of S-box transition probabilities. It is standard to work with

$$\text{NegLog}(\mathcal{T}) = -\log_2 \Pr[\mathcal{T}],$$

which avoids underflow and linearizes multiplicative costs. Lower NegLog indicates a more likely trail [23].

Nodes expanded. In branch-and-bound or beam search, the number of expanded nodes proxies the computational effort: each node is a partial trail extended by feasible transitions. Reducing nodes without losing the optimum is a key acceleration target [28].

Wall-clock time. Runtime reflects practical performance beyond node counts, incorporating implementation details and hardware. To ensure fairness, we evaluate under identical resources and parameter settings.

All cost metrics summarize in Table 4.

Table 4: Summary of cost metrics used in differential trail search.

Metric	Meaning	Usage
NegLog	$-\log_2 \Pr[\mathcal{T}]$	Quality (trail likelihood)
Nodes expanded	Count of explored partial trails	Proxy for computational effort
Wall-clock time	Real runtime of the search	Practical efficiency benchmark

2.4 Search Algorithms

Branch-and-bound (BnB). Branch-and-bound systematically explores the search tree while maintaining global upper/lower bounds to prune any subtree that is provably worse than the best known solution. It is *complete* (given sufficient time and memory) and can provide optimality guarantees. However, in very large combinatorial spaces, the overhead of maintaining large queues and globally updating bounds often becomes the main bottleneck in practice [18].

Beam search (BS). Beam search is a *width-limited, greedy* tree search. At each depth (round), it scores all newly expanded states and retains only the top **beam** states, discarding the rest. Unlike BnB, beam search is not complete and does not guarantee optimality; its effectiveness hinges on how well the scoring function correlates with the true objective. In return, it offers predictable memory use and good practical throughput [24].

Scoring in differential trail search. Let the frontier at round r contain states (Δ^r, W^r) , where W^r is the accumulated negative log-probability (NegLog) up to round r . A standard score adds a *lower bound* (LB) on the remaining cost for $h = D - r$ rounds:

$$\text{Score}(\Delta^r) = W^r + \text{LB}(\Delta^r, h).$$

For SPN ciphers with 4-bit S-boxes, a common surrogate is

$$\text{LB}(\Delta^r, h) = \text{HW}_{\text{nib}}(\Delta^r) \cdot h,$$

where HW_{nib} counts active (nonzero) nibbles in Δ^r . This linear LB is inexpensive to compute and provides a consistent ordering signal.

Pruning rules. After scoring the candidates at depth $r + 1$, one typically applies: (i) α -pruning, which keeps all states with $\text{Score} \leq \alpha \cdot \text{Score}_{\min}$ for a fixed $\alpha > 1$; or (ii) a *percentile* rule, which keeps the top fraction (e.g., top 25%). A final *beam cap* trims the list to at most **beam** states for the next round. Some implementations also keep a small number of *lifeboats*—states with the lowest accumulated W^r regardless of total score—to maintain diversity and reduce the risk of early commitment to a single branch.

Trade-offs. BnB favors completeness and provable optimality but can be memory- and time-intensive. Beam search trades these guarantees for predictable memory usage and speed, and is particularly attractive when a cheap, round-wise LB yields a ranking that correlates reasonably well

with the final trail cost. In cryptanalytic settings, this balance often enables deeper explorations within fixed resource budgets.

Section 3 will introduce a learned *residual* correction to this LB that preserves the existing beam and pruning parameters while aiming to improve the quality of ranking.

3 Proposed Method

Our goal is to improve the *ranking* of partial trails inside a fixed-width beam search by correcting the classic lower bound (LB) on the remaining cost. We do not change beam size, pruning rules, or any safety parameters; the only difference is how the tail cost is estimated for scoring. This section describes (i) the setting and notation, (ii) how we learn a residual correction to the LB, and (iii) how we fuse it back at inference while preserving safety.

3.1 Setting and Notation

Let D be the total number of rounds and r the current round. Each frontier element is a pair (Δ^r, W^r) , where Δ^r is the state difference after round r and W^r is the accumulated negative log-probability (NegLog). With $h = D - r$ rounds left, the classic score minimized by the beam is

$$\text{Score}_{\text{classic}}(\Delta^r) = W^r + \text{LB}(\Delta^r, h),$$

where, for 4-bit S-box SPNs, a standard surrogate is

$$\text{LB}(\Delta^r, h) = \text{HW}_{\text{nib}}(\Delta^r) \cdot h,$$

and HW_{nib} counts active (nonzero) nibbles. This LB is efficient and safe but often under-discriminative: many nodes receive very similar scores even though their true tails differ significantly.

3.2 Learning a Residual to the Lower Bound

Instead of predicting the absolute tail cost, we learn the *residual* error of the LB:

$$\text{resid}(\Delta^r, h) := \text{Tail}^*(\Delta^r, h) - \text{LB}(\Delta^r, h),$$

where Tail^* is an offline estimate of the best tail cost from Δ^r (obtained by short, targeted searches). Regressing residuals focuses the model on systematic imperfections of the LB and empirically yields more stable generalization than direct tail regression.

Training data in brief. We generate supervised pairs by expanding two rounds from random inputs to an intermediate difference Δ^{mid} , then chaining multi-horizon tail searches from Δ^{mid} for $h \in \{2, 8, 14, 18\}$. Each record includes one-hot nibble encodings of $(\Delta^{\text{in}}, \Delta^{\text{mid}}, \Delta^{\text{out}})$ plus light scalar features $[\text{HW}_{\text{in}}, \text{HW}_{\text{mid}}, \text{HW}_{\text{out}}, W^{(\text{mid})}, h]$, with label *resid*. We train a histogram gradient boosting regressor (HGB) with early stopping. To avoid leakage, we group the train/validation split by $(\Delta^{\text{in}}, \Delta^{\text{out}}, h)$ so that all records sharing the same endpoints and horizon fall on the same side of the split.

Per-horizon calibration. Because pruning depends on ranking quality rather than absolute scale, we optionally fit an isotonic map g_h on the validation set (only when enough samples exist) to monotonically refine residual predictions for each horizon h . At inference time we use g_h if available, or, if h was unseen, the nearest $g_{h'}$.

3.3 Inference-Time Fusion (Single Default)

At round r with $h = D - r$, for each expanded candidate we predict a residual $\widehat{\text{resid}}$, apply g_h if available, and then *robustly* center and scale these predictions within the current layer using the median med and a scale $\sigma \approx \text{IQR}/1.349$ (with a small floor to avoid division by zero). We then form a *multiplicative, saturating* correction:

$$\widehat{\text{Tail}}(\Delta^r, h) = \text{LB}(\Delta^r, h) \cdot \left(1 + g \cdot \tanh((\widehat{\text{resid}} - \text{med})/\sigma)\right), \quad (4)$$

with a small gain g (e.g., 0.25–0.5). This design applies *relative* adjustments (stronger where LBs are larger), improves ordering when the layer has an almost constant offset, and gracefully saturates against outliers.

For safety we enforce a multiplicative floor:

$$\widehat{\text{Tail}} \leftarrow \max(\text{lb_floor} \cdot \text{LB}(\Delta^r, h), \widehat{\text{Tail}}), \quad \text{lb_floor} \in [0.5, 0.6], \quad (5)$$

which prevents overly optimistic underestimation of the tail. Other fusion strategies (e.g., simple additive corrections) are possible but are not explored here.

3.4 Integration with the Beam (Early-Round Use)

The search loop remains identical to the classic beam search except for the scoring line in the first L rounds. After expanding candidates and deduplicating identical (Δ^{r+1}, W^{r+1}) pairs, we compute scores as

$$\text{Score} = W^{r+1} + \begin{cases} \widehat{\text{Tail}}(\Delta^{r+1}, D - (r+1)) & \text{if } r+1 \leq L, \\ \text{LB}(\Delta^{r+1}, D - (r+1)) & \text{otherwise.} \end{cases}$$

We then apply the same α -rule and optional lifeboats as in the classic method, and finally cap by the beam width. A concise pseudocode sketch is given in Algorithm 1.

Algorithm 1 ML-guided beam search with early-round fusion

Require: total rounds D , beam size B , early model depth L , residual model f , per- h isotonic maps $\{g_h\}$ (optional), floor lb_floor

- 1: Initialize frontier $\mathcal{F}_0 \leftarrow \{(\Delta^0, W^0=0)\}$
- 2: **for** $r = 0$ **to** $D - 1$ **do**
- 3: $h \leftarrow D - r$
- 4: // **expand** each $(\Delta^r, W^r) \in \mathcal{F}_r$ to all feasible (Δ^{r+1}, W^{r+1})
- 5: // **deduplicate** identical Δ^{r+1} by keeping the smallest W^{r+1}
- 6: **if** $r+1 \leq L$ **then**
- 7: form features X for all unique Δ^{r+1} and compute $\widehat{\text{resid}} \leftarrow f(X)$
- 8: **if** g_h exists: $\widehat{\text{resid}} \leftarrow g_h(\widehat{\text{resid}})$
- 9: compute $\text{med} \leftarrow \text{median}(\widehat{\text{resid}})$ and $\sigma \leftarrow \max(\varepsilon, \text{IQR}/1.349)$
- 10: **for** each unique Δ^{r+1} **do**
- 11: $\text{LB} \leftarrow \text{LB}(\Delta^{r+1}, h-1)$
- 12: $z \leftarrow (\widehat{\text{resid}} - \text{med})/\sigma$
- 13: $\widehat{\text{Tail}} \leftarrow \text{LB} \cdot (1 + g \cdot \tanh(z))$ $\triangleright$ Eq. (4)
- 14: $\widehat{\text{Tail}} \leftarrow \max(\text{lb_floor} \cdot \text{LB}, \widehat{\text{Tail}})$ $\triangleright$ Eq. (5)
- 15: $\text{Score}(\Delta^{r+1}) \leftarrow W^{r+1} + \widehat{\text{Tail}}$
- 16: **end for**
- 17: **else**
- 18: $\text{Score}(\Delta^{r+1}) \leftarrow W^{r+1} + \text{LB}(\Delta^{r+1}, h-1)$
- 19: **end if**
- 20: // **prune** by α -rule around $\min \text{Score}$ (with optional cap) and add a small lifeboats set by low W^{r+1}
- 21: $\mathcal{F}_{r+1} \leftarrow \text{top-}B \text{ states by Score}$
- 22: **end for**
- 23: **return** best (Δ^D, W^D) in $\mathcal{F}_D$

3.5 Complexity and Safety

S-box expansions—the dominant cost—are identical to the classic search. Model-side overhead consists of a single batched prediction per round where the model is active, plus per-layer median/IQR statistics. This adds a small constant factor to early rounds and is typically offset by stronger pruning. We do not relax α or enlarge the beam, so any reduction in nodes or time comes purely from better ordering. The floor in (5) preserves the classic method’s safety margin by preventing tails from dropping too far below the original LB.

4 Experimental Evaluation

All implementations are written in Python 3.11. All experiments are run on a single machine with an Intel Core i7-8550U at 1.80 GHz and 11 GB RAM under Debian 12. No GPU acceleration is used. For all results below, the classic and ML-guided beam searches share the same code base and are executed under identical conditions; only the scoring heuristic differs.

Table 5: Recovered 12-round GIFT-64 trails vs. published characteristics.

Source (table)	Published NegLog	Our NegLog	Path match
Zhu et al. [36], Table 4	59	59	yes
Zhu et al. [36], Table 6	60	60	yes
Chen et al. [8], Table 3	58	58	yes

Table 6: Search cost for the recovered 12-round trails.

Source (table)	Nodes expanded	Time (s)
Zhu et al. [36], Table 4	117,801	4.48
Zhu et al. [36], Table 6	151,793	6.49
Chen et al. [8], Table 3	165,240	5.95

4.1 Reproducing Published 12-Round GIFT-64 Trails

As a sanity check, we verify that our search can recover known high-probability 12-round trails for GIFT-64 from the literature. We take three representative input/output-difference pairs (Δ^0, Δ^{12}) . Concretely, we use the following pairs:

- Zhu et al. [36], Table 4: $\Delta^0 = 0x0c00000000600000$, $\Delta^{12} = 0x1008000000022000$.
- Zhu et al. [36], Table 6: $\Delta^0 = 0x0000000000001010$, $\Delta^{12} = 0x0000000000001010$.
- Chen et al. [8], Table 3: $\Delta^0 = 0x0000000600000006$, $\Delta^{12} = 0x0000000500000005$.

In all three cases, our search finds a 12-round trail whose total weight (NegLog) exactly matches the published value (59, 60, and 58, respectively), and the sequence of intermediate differences $\{\Delta^r\}_{r=0}^{12}$ coincides round by round with the characteristics listed in the corresponding tables. This shows that our implementation and scoring conventions are consistent with prior work and that the ML-guided scoring does not degrade trail quality (Table 5). We also report nodes and runtime as an indication of search cost on these non-trivial instances (Table 6); however, due to different platforms and solvers, these timings are not intended for direct comparison with MILP-based results.

We evaluate on GIFT-64 under identical search settings unless noted: $\alpha = 1.08$, `beam`= 4096, $t = 3$, `per_nib_cap`= 8, $\alpha_{\text{cap}} = 0.5$, `lifeboats_schedule`="32,32,48,64,96,128,192,256", and `lb_floor_scale`= 0.6. When the model is used, we employ the multiplicative `mult_tanh` fusion with gain $g = 0.5$ and per- h isotonic calibration enabled. We report the number of expanded nodes, wall-clock time, and the best trail weight (NegLog). Unless stated otherwise, the residual model is the baseline one trained on our original dataset; in one ablation we also use a variant trained on an expanded dataset ("extended model").

After confirming in Section 4.1 that our implementation reproduces known 12-round trails from the literature with identical NegLog and round-by-round differences, we now examine how the ML-guided scoring affects search cost across different depths.

4.2 Low-Round Behaviour at $D=8$

We first fix the pair $\Delta_{\text{in}} = 0x0000000600000006$ and $\Delta_{\text{out}} = 0x0000000500000005$ over $D = 8$ rounds and compare the classic beam search with several ML-guided variants. Table 7 summarizes

Table 7: Low-round search at $D=8$ for $\Delta_{\text{in}} = 0\text{x}0000000600000006$, $\Delta_{\text{out}} = 0\text{x}0000000500000005$. “Baseline” and “Extended” denote the two residual models (original vs. larger training set).

Mode	Model	L (use_model_up_to)	Expanded nodes	Time [s]	Best NegLog
Classic	–	–	38,364	0.721	38.0
ML	Baseline	3	38,044	1.625	38.0
ML	Baseline	4	22,328	1.470	38.0
ML	Baseline	6	22,328	2.213	38.0
ML	Extended	4	22,168	1.427	38.0

Table 8: Medium-depth search at $D=10$ for $\Delta_{\text{in}} = 0\text{x}0000000600000006$, $\Delta_{\text{out}} = 0\text{x}0000000500000005$.

Mode	L	Expanded nodes	Time [s]	Best NegLog
Classic	–	76,589	1.724	48.0
ML (baseline)	6	76,239	4.037	48.0

the results.

Across all configurations the best NegLog remains 38, and the full trail coincides with the classic search. Using the baseline model for the first $L=4$ rounds trims the number of expanded nodes by about 42% relative to the classic beam, showing that the learned tail estimate produces a sharper ranking even in shallow searches. However, at this depth the classic search is already very cheap, so model inference dominates: runtimes roughly double compared to classic. Using the model for too few rounds ($L=3$) gives almost no pruning, while extending it further ($L=6$) leaves the node count unchanged but increases time. The extended model (trained on a larger dataset) yields a slightly smaller node count and marginally lower time at $L=4$, illustrating that a richer training set can bring incremental gains.

4.3 Medium-Depth Searches at $D=10$ and $D=12$

We next consider the same input pair $\Delta_{\text{in}} = 0\text{x}0000000600000006$ with two different output patterns and depths, $D=10$ and $D=12$.

$D=10$, $\Delta_{\text{out}} = 0\text{x}0000000500000005$. Table 8 compares the classic beam and an ML-guided variant with $L=6$.

Here the ML-guided beam explores essentially the same number of nodes as the classic search, with strictly higher runtime due to model overhead, while the best NegLog and full trail are identical. This instance illustrates that the residual model does not uniformly improve pruning: when the classic LB already ranks states well for a given configuration, the learned correction has little effect.

$D=12$, $\Delta_{\text{out}} = 0\text{x}0000000500000005$. For $D=12$ we return to the pair from Section 4.1 (also listed in Table 3 of [8]) and vary the model horizon L . Results appear in Table 9.

All configurations recover the same best trail with NegLog 58. For moderate horizons ($L=6$ or 8), the node count is within 1% of the classic search and runtime is higher, again reflecting model overhead. Driving the model through all 12 rounds reduces nodes by about 9%, but at the cost

Table 9: Medium-depth search at $D=12$ for $\Delta_{\text{in}} = 0x0000000600000006$, $\Delta_{\text{out}} = 0x0000000500000005$.

Mode	L	Expanded nodes	Time [s]	Best NegLog
Classic	–	166,130	3.525	58.0
ML (baseline)	6	165,536	5.997	58.0
ML (baseline)	8	166,124	7.536	58.0
ML (baseline)	12	151,038	19.654	58.0

Table 10: Higher-depth search at $D=13$ for $\Delta_{\text{in}} = 0x00c0000000c00000$, $\Delta_{\text{out}} = 0x00000000000001010$.

Mode	L	Expanded nodes	Time [s]	Best NegLog
Classic	–	299,441	5.643	64.0
ML (baseline)	4	261,479	5.776	64.0
ML (baseline)	6	259,213	7.078	64.0
ML (baseline)	11	252,667	17.456	64.0

of a large slowdown. This confirms the trade-off between additional per-layer ranking information and the constant per-round inference cost.

4.4 Higher-Depth Search at $D=13$

We now move to $D=13$ rounds and a different difference pair, $\Delta_{\text{in}} = 0x00c0000000c00000$ and $\Delta_{\text{out}} = 0x00000000000001010$. This pair corresponds to a 13-round trail that extends one of the 12-round characteristics reproduced earlier. Table 10 compares the classic beam with ML-guided runs at different horizons L .

For $D=13$, ML-guided scoring provides clearer pruning benefits. Using the model in the first $L=4$ rounds reduces the number of expanded nodes by roughly 13% with a runtime very close to classic. Extending the horizon to $L=6$ yields a slightly larger node reduction (about 13–14%) but with a noticeable slowdown. Pushing the model much deeper ($L=11$) trims around 16% of the nodes but more than triples runtime. Again, all variants recover the same best trail (NegLog 64).

4.5 Deep Search at $D=16$ with the Extended Model

Finally, we consider a deeper search with $D=16$ rounds using the same difference pair as in the 8/12-round experiments, $\Delta_{\text{in}} = 0x0000000600000006$, $\Delta_{\text{out}} = 0x0000000500000005$, and the extended residual model trained on the larger dataset. Table 11 shows classic vs. ML-guided runs for several horizons L .

At $D=16$ the search space is large enough that the ML-guided beam remains comparable in node count to the classic search for small horizons ($L=4$), and progressively reduces nodes as L grows: roughly 5%, 7%, and 12% fewer expanded nodes for $L=8$, 12, and 16, respectively. However, the overhead of repeatedly calling the model becomes dominant, so runtimes increase monotonically with L . Even in this deep setting, the best NegLog and full trail remain identical across all runs.

Summary. Across all depths and parameter choices considered, the ML-guided beam search consistently preserves the best trail quality: in every experiment it finds the same best NegLog (and the same trail) as the classic beam under identical pruning parameters. The main observable

Table 11: Deep search at $D=16$ for $\Delta_{\text{in}} = 0\text{x}0000000600000006$, $\Delta_{\text{out}} = 0\text{x}0000000500000005$, using the extended residual model.

Mode	L	Expanded nodes	Time [s]	Best NegLog
Classic	–	615,430	10.121	78.0
ML (extended)	4	615,150	11.435	78.0
ML (extended)	8	583,420	14.332	78.0
ML (extended)	12	575,086	23.232	78.0
ML (extended)	16	543,682	44.229	78.0

effect is on search cost: for shallow depths ($D=8-10$), the model can significantly reduce the number of expanded nodes, but the classic search is already fast enough that inference overhead dominates. For medium and higher depths ($D=12-16$), moderate horizons L yield node reductions on the order of 10–15% with runtimes close to classic, while overly aggressive use of the model trims slightly more nodes at the price of substantial slowdown. These experiments confirm that the proposed residual correction improves the ranking inside the beam without sacrificing trail optimality, and that its practical benefit depends on depth and on how far into the search the model is invoked.

5 Conclusion

This work shows that a small learned residual correction to a simple nibble-count lower bound can be plugged into a standard beam search for GIFT-64 without changing beam/pruning parameters, while reproducing known optimal trails and reducing node expansions at moderate depths.

A natural next step is to apply the same recipe to other 4-bit SPNs such as PRESENT, GIFT-128, or RECTANGLE, to see how much the benefit depends on the S-box and diffusion layer. On the model side, lighter regressors or alternative feature sets could further reduce the inference overhead while preserving the ranking quality. Finally, integrating such residual heuristics with other search back-ends (BnB, MILP, SAT) or with key-dependent probability refinements is an interesting direction, but lies beyond the scope of the present work.

We studied whether a learned heuristic can safely improve differential trail search within the classical Markov model. Instead of predicting absolute tail costs, we learn residuals to a cheap nibble-count lower bound and fuse them multiplicatively with robust centering and a conservative floor, so that beam width, α -pruning, and lifeboat rules remain unchanged.

On GIFT-64, the ML-guided beam search reproduces the best-known trail weights (and full intermediate differences) reported in prior work, while in several settings it reduces the number of expanded nodes by about 10–40% at comparable parameters. For very shallow searches the classic heuristic remains competitive and model overhead dominates, but at medium depths our residual-guided scoring yields a more favorable trade-off between pruning strength and runtime. This suggests that even relatively simple ML models can act as drop-in improvements to existing cryptanalytic search heuristics, without altering the underlying algorithms or probability model.

References

- [1] Subhadeep Banik, Sumit Kumar Pandey, Thomas Peyrin, Yu Sasaki, Siang Meng Sim, and Yosuke Todo. Gift: A small present: Towards reaching the limit of lightweight encryption. In *Cryptographic Hardware and Embedded Systems—CHES 2017: 19th International Conference, Taipei, Taiwan, September 25–28, 2017, Proceedings*, pages 321–345. Springer, 2017.

- [2] Tim Beyne and Vincent Rijmen. Differential cryptanalysis in the fixed-key model. In *Annual International Cryptology Conference*, pages 687–716. Springer, 2022.
- [3] Eli Biham and Adi Shamir. Differential cryptanalysis of des-like cryptosystems. *Journal of CRYPTOLOGY*, 4(1):3–72, 1991.
- [4] Alex Biryukov and Vesselin Velichkov. Automatic search for differential trails in arx ciphers. In *Cryptographers’ track at the RSA conference*, pages 227–250. Springer, 2014.
- [5] Andrey Bogdanov, Lars R Knudsen, Gregor Leander, Christof Paar, Axel Poschmann, Matthew JB Robshaw, Yannick Seurin, and Charlotte Vikkelsoe. Present: An ultra-lightweight block cipher. In *Cryptographic Hardware and Embedded Systems-CHES 2007: 9th International Workshop, Vienna, Austria, September 10-13, 2007. Proceedings 9*, pages 450–466. Springer, 2007.
- [6] Federico Canale and María Naya-Plasencia. Guessing less and better: improved attacks on gift-64. *Designs, Codes and Cryptography*, 93(3):787–822, 2025.
- [7] Anne Canteaut, Eran Lambooi, Samuel Neves, Shahram Rasoolzadeh, Yu Sasaki, and Marc Stevens. Refined probability of differential characteristics including dependency between multiple rounds. *IACR Transactions on Symmetric Cryptology*, 2017(2):203–227, 2017.
- [8] Huaifeng Chen, Rui Zong, and Xiaoyang Dong. Improved differential attacks on gift-64. In *International Conference on Information and Communications Security*, pages 447–462. Springer, 2019.
- [9] Lele Chen, Gaoli Wang, and GuoYan Zhang. Milp-based related-key rectangle attack and its application to gift, khudra, mibs. *The Computer Journal*, 62(12):1805–1821, 2019.
- [10] Yincen Chen, Nana Zhang, Xuanyu Liang, Ling Song, Qianqian Yang, and Zhuohui Feng. Improving the rectangle attack on gift-64. In *International Conference on Selected Areas in Cryptography*, pages 43–61. Springer, 2023.
- [11] Yaxin Cui, Hong Xu, and Wenfeng Qi. Milp-based linear attacks on round-reduced gift. *Chinese Journal of Electronics*, 31(1):89–98, 2022.
- [12] Yaxin Cui, Hong Xu, Lin Tan, Huajin Chen, and Wenfeng Qi. Construction of equivalent linear trails and multiple linear attack on reduced-round gift-64. *Journal of Cryptographic Engineering*, 13(2):223–234, 2023.
- [13] Joan Daemen and Vincent Rijmen. Plateau characteristics. *IET information security*, 1(1): 11–17, 2007.
- [14] Christophe De Canniere and Christian Rechberger. Finding sha-1 characteristics: General results and applications. In *International conference on the theory and application of cryptology and information security*, pages 1–20. Springer, 2006.
- [15] Muhammad ElSheikh, Ahmed Abdelkhalek, and Amr M Youssef. On milp-based automatic search for differential trails through modular additions with application to bel-t. In *International Conference on Cryptology in Africa*, pages 273–296. Springer, 2019.
- [16] Fulei Ji, Wentao Zhang, and Tianyou Ding. Improving matsui’s search algorithm for the best differential/linear trails and its applications for des, desl and gift. *The Computer Journal*, 64(4):610–627, 2021.

- [17] Xuejia Lai, James L Massey, and Sean Murphy. Markov ciphers and differential cryptanalysis. In *Workshop on the Theory and Application of Cryptographic Techniques*, pages 17–38. Springer, 1991.
- [18] A. H. Land and A. G. Doig. An automatic method of solving discrete programming problems. *Econometrica*, 28(3):497–520, 1960.
- [19] Gaëtan Leurent. Analysis of differential attacks in arx constructions. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 226–243. Springer, 2012.
- [20] Yingying Li and Qichun Wang. The sat-based automatic searching and experimental verification for differential characteristics with application to midori64. In *International Conference on Information Security*, pages 153–161. Springer, 2022.
- [21] Fukang Liu, Takanori Isobe, and Willi Meier. Automatic verification of differential characteristics: Application to reduced gimli. In *Annual International Cryptology Conference*, pages 219–248. Springer, 2020.
- [22] Yunwen Liu, Wenying Zhang, Bing Sun, Vincent Rijmen, Guoqiang Liu, Chao Li, Shaojing Fu, and Meichun Cao. The phantom of differential characteristics. *Designs, Codes and Cryptography*, 88(11):2289–2311, 2020.
- [23] Zhengbin Liu, Yongqiang Li, and Mingsheng Wang. Optimal differential trails in simon-like ciphers. *Cryptology ePrint Archive*, 2017.
- [24] Bruce T. Lowerre. *The HARPY Speech Recognition System*. PhD thesis, Carnegie Mellon University, 1976.
- [25] Nicky Mouha, Qingju Wang, Dawu Gu, and Bart Preneel. Differential and linear cryptanalysis using mixed-integer linear programming. In *International Conference on Information Security and Cryptology*, pages 57–76. Springer, 2011.
- [26] Marcel Nageler, Shibam Ghosh, Marlene Jüttler, and Maria Eichlseder. Autodiver: Automatically verifying differential characteristics and learning key conditions. *Cryptology ePrint Archive*, 2025.
- [27] Thomas Peyrin and Quan Quan Tan. Mind your path: On (key) dependencies in differential characteristics. *Cryptology ePrint Archive*, 2022.
- [28] Thomas Peyrin, Quan Quan Tan, Hongyi Zhang, and Chunming Zhou. Trail-estimator: An automated verifier for differential trails in block ciphers. *Cryptology ePrint Archive*, 2025.
- [29] Ling Sun. A linearisation method for identifying dependencies in differential characteristics: Examining the intersection of deterministic linear relations and nonlinear constraints. *Cryptology ePrint Archive*, 2024.
- [30] Ling Sun, Wei Wang, and Meiqin Wang. More accurate differential properties of led64 and midori64. *IACR Transactions on Symmetric Cryptology*, pages 93–123, 2018.
- [31] Ling Sun, Wei Wang, and Meiqin Wang. Improved attacks on gift-64. In *International Conference on Selected Areas in Cryptography*, pages 246–265. Springer, 2021.

- [32] Siwei Sun, Lei Hu, Peng Wang, Kexin Qiao, Xiaoshuang Ma, and Ling Song. Automatic security evaluation and (related-key) differential characteristic search: application to simon, present, lblock, des (l) and other bit-oriented block ciphers. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 158–178. Springer, 2014.
- [33] Jingsui Weng, Wentao Zhang, Ting Peng, and Tianyou Ding. Improving the search algorithm for the best differential/linear trails of bit-permutation-based ciphers. In *International Conference on Information Security and Cryptology*, pages 301–321. Springer, 2024.
- [34] Wei-Zhu Yeoh, Je Sen Teh, and Jiageng Chen. Automated search for block cipher differentials: A gpu-accelerated branch-and-bound algorithm. In *Australasian Conference on Information Security and Privacy*, pages 160–179. Springer, 2020.
- [35] Chunming Zhou, Wentao Zhang, Tianyou Ding, and Zejun Xiang. Improving the milp-based security evaluation algorithm against differential/linear cryptanalysis using a divide-and-conquer approach. *Cryptology ePrint Archive*, 2019.
- [36] Baoyu Zhu, Xiaoyang Dong, and Hongbo Yu. Milp-based differential attack on round-reduced gift. In *Cryptographers’ track at the RSA conference*, pages 372–390. Springer, 2019.